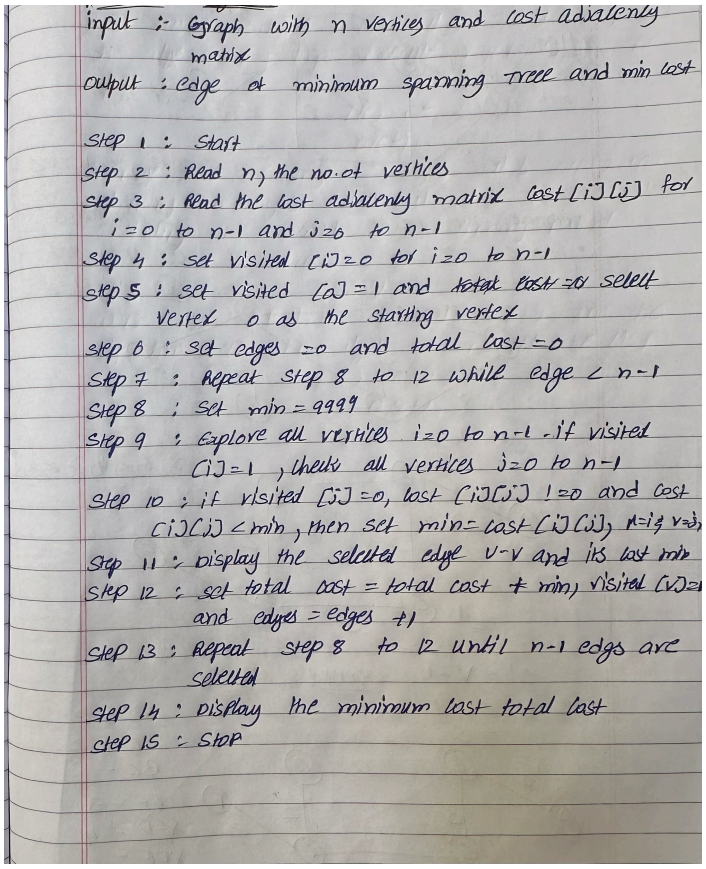
PRACTICAL - 9

AIM : Implement prim's algorithm.

ALGORITHM :



PROGRAM :

```
#include <iostream>
using namespace std;

class Graph
{
public:
    int cost[10][10];
    int visited[10];
    int n;

    void getInput()
    {
        cout << "Enter number of vertices: ";
```

```
cin >> n;

    cout << "Enter cost adjacency matrix:\n";

    for (int i = 0; i < n; i++)
    {
       for (int j = 0; j < n; j++)
       {
          cin >> cost[i][j];
       }
    }
}

 void prim()
{
    int edges = 0;
    int min;
    int u = 0, v = 0;
    int totalCost = 0;

    for (int i = 0; i < n; i++)
    {
       visited[i] = 0;
    }

    visited[0] = 1;

    cout << "\nEdges in Minimum Spanning Tree:\n";

    while (edges < n - 1)
    {
       min = 9999;

       for (int i = 0; i < n; i++)
       {
          if (visited[i] == 1)
          {
             for (int j = 0; j < n; j++)
             {
                if (visited[j] == 0 && cost[i][j] != 0)
                {
                   if (cost[i][j] < min)
                   {
```

```
                min = cost[i][j];
                        u = i;
                        v = j;

                }
                    }
                }
                }
            }

            if (min == 9999)
            {
                cout << "Graph is disconnected. MST cannot be formed.\n";
                return;
            }

            cout << u << " - " << v << " = " << min << endl;

            totalCost += min;
            visited[v] = 1;
            edges++;
        }

        cout << "\nMinimum Cost = " << totalCost << endl;
    }
};

int main()
{
    Graph g;

    g.getInput();
    g.prim();

    return 0;
}
```

OUTPUT:

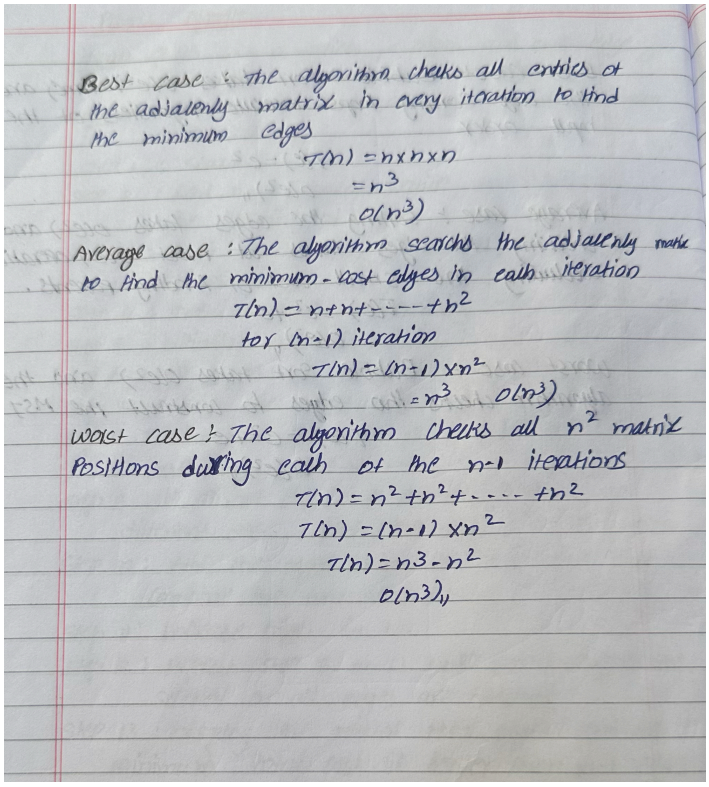
```
Output

Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0

Edges in Minimum Spanning Tree:
0 - 3 = 5
3 - 2 = 4
0 - 1 = 10

Minimum Cost = 19
```

TIME COMPLEXITY :



CONCLUSION :

I successfully implemented Prim's Algorithm to find the Minimum Spanning Tree (MST) of a weighted, undirected graph. I learned how to select the minimum-cost edges while avoiding cycles and connect all vertices with the minimum total cost. Through this practical, I understood the working of a greedy algorithm and its application in finding an optimal solution. I also learned to represent a graph using an adjacency matrix and calculate the minimum cost of the spanning tree.